Java面试题大全

目录

1. Java基础
2. 集合框架
3. 多线程与并发
4. JVM原理
5. Spring框架
6. 数据库与ORM
7. 设计模式
8. 系统设计
9. 编程题

1. Java基础

1.1 基本概念

Q1: Java有哪些特点？

- 面向对象：封装、继承、多态
- 平台无关性：通过JVM实现"一次编写，到处运行"
- 简单易学：语法简洁，自动内存管理
- 安全性：无指针操作，字节码验证
- 多线程：内置多线程支持

Q2: JDK、JRE、JVM的区别是什么？

- JDK：Java开发工具包，包含JRE和开发工具
- JRE：Java运行环境，包含JVM和核心类库
- JVM：Java虚拟机，负责执行字节码

1.2 数据类型

Q3: Java的基本数据类型有哪些？

- 整数类型：byte(1字节)、short(2字节)、int(4字节)、long(8字节)
- 浮点类型：float(4字节)、double(8字节)
- 字符类型：char(2字节)
- 布尔类型：boolean

Q4: ==和equals的区别？

- ==：比较基本数据类型的值或对象的引用地址
- equals：比较对象的内容（需要重写）

1.3 面向对象

Q5: 什么是封装？
封装是将数据（属性）和操作数据的方法绑定在一起，通过访问修饰符控制访问权限。

Q6: 什么是继承？
继承是子类继承父类的属性和方法，实现代码复用。

Q7: 什么是多态？
多态是指同一个方法在不同对象中有不同的实现方式，包括方法重载和方法重写。

2. 集合框架

2.1 集合体系

Q8: Java集合框架的主要接口有哪些？

- Collection接口
  - List接口：ArrayList、LinkedList
  - Set接口：HashSet、TreeSet
- Map接口：HashMap、TreeMap

2.2 ArrayList vs LinkedList

Q9: ArrayList和LinkedList的区别？

特性	ArrayList	LinkedList
底层结构	数组	双向链表
随机访问	O(1)	O(n)
插入删除	O(n)	O(1)
内存占用	较少	较多

2.3 HashMap原理

Q10: HashMap的工作原理？

- 基于哈希表实现
- 使用数组+链表/红黑树结构
- 通过hashCode()计算哈希值
- 通过equals()解决哈希冲突
- JDK8后链表长度>8时转换为红黑树

3. 多线程与并发

3.1 线程创建

Q11: 创建线程的几种方式？

1. 继承Thread类
2. 实现Runnable接口
3. 实现Callable接口

4. 使用线程池

3.2 线程状态

Q12: 线程的生命周期状态？

- 新建（NEW）
- 运行（RUNNABLE）
- 阻塞（BLOCKED）
- 等待（WAITING）
- 超时等待（TIMED_WAITING）
- 终止（TERMINATED）

3.3 并发工具

Q13: synchronized和ReentrantLock的区别？

特性	synchronized	ReentrantLock
实现方式	JVM层面	API层面
等待可中断	否	是
公平锁	否	可设置
条件变量	wait/notify	Condition

4. JVM原理

4.1 内存结构

Q14: JVM内存区域划分？

- 方法区：存储类信息、常量、静态变量
- 堆：存储对象实例
- 栈：存储局部变量、方法调用
- 程序计数器：记录当前线程执行位置
- 本地方法栈：Native方法调用

4.2 垃圾回收

Q15: 常见的垃圾回收算法？

- 标记-清除算法
- 复制算法
- 标记-整理算法
- 分代收集算法

Q16: Minor GC和Full GC的区别？

- Minor GC：发生在新生代，频率高，时间短
- Full GC：发生在老年代，频率低，时间长

5. Spring框架

5.1 Spring核心

Q17: Spring框架的核心特性？

- IOC（控制反转）
- DI（依赖注入）
- AOP（面向切面编程）
- 事务管理
- MVC框架

5.2 IOC容器

Q18: Bean的作用域有哪些？

- singleton：单例（默认）
- prototype：原型
- request：请求级别
- session：会话级别
- application：应用级别

5.3 AOP原理

Q19: Spring AOP的实现原理？

- 基于动态代理实现
- JDK动态代理：基于接口
- CGLIB代理：基于子类
- 通过代理对象实现切面逻辑

6. 数据库与ORM

6.1 JDBC

Q20: JDBC编程的步骤？

1. 加载数据库驱动
2. 建立数据库连接
3. 创建Statement对象
4. 执行SQL语句
5. 处理结果集
6. 关闭连接

6.2 MyBatis

Q21: MyBatis的优点？

- 简化JDBC操作
- 支持动态SQL

- 提供缓存机制
- 易于集成Spring
- SQL与代码分离

7. 设计模式

7.1 创建型模式

Q22: 单例模式的实现方式？

1. 饿汉式（线程安全）
2. 懒汉式（线程不安全）
3. 双重检查锁（推荐）
4. 静态内部类（推荐）
5. 枚举（最安全）

7.2 结构型模式

Q23: 代理模式的应用场景？

- 远程代理
- 虚拟代理
- 保护代理
- 智能引用代理

7.3 行为型模式

Q24: 观察者模式的应用？

- 事件处理系统
- 消息订阅机制
- MVC架构中的Model-View关系

8. 系统设计

8.1 缓存设计

Q25: 缓存穿透、缓存击穿、缓存雪崩？

- 缓存穿透：查询不存在的数据，解决方案：布隆过滤器
- 缓存击穿：热点key过期，解决方案：互斥锁、永不过期
- 缓存雪崩：大量key同时过期，解决方案：随机过期时间

8.2 分布式系统

Q26: CAP理论？

- Consistency（一致性）
- Availability（可用性）

- Partition tolerance（分区容错性）
- 三者只能满足其二

9. 编程题

9.1 算法题

Q27: 实现二分查找

```
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

Q28: 单例模式实现（双重检查锁）

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

9.2 多线程题

Q29: 生产者消费者模型

```
public class ProducerConsumer {
    private final Queue<Integer> queue = new LinkedList<>();
    private final int MAX_SIZE = 10;

    public synchronized void produce() throws InterruptedException {
        while (queue.size() == MAX_SIZE) {
            wait();
        }
        queue.add(1);
        notifyAll();
    }

    public synchronized void consume() throws InterruptedException {
        while (queue.isEmpty()) {
            wait();
        }
```

```
        queue.poll();
        notifyAll();
    }
}
```

总结

这份Java面试题文档涵盖了从基础到高级的各个知识点，适合作为RAG系统的知识库。建议在实际使用时：

1. 根据具体技术栈调整内容深度
2. 定期更新最新技术面试题
3. 结合实际项目经验进行补充
4. 关注新兴技术趋势

通过系统学习和实践，相信能够帮助开发者在面试中脱颖而出。

（AI生成）